

Unit-2

Internet Address

2.1 The InetAddress Class

- The InetAddress class in Java represents an internet protocol (IP) address.
- It allows the program to work with IP addresses (both IPv4 and IPv6) and DNS names.

Creating New InetAddress Objects

- InetAddress objects are not created directly. Instead, they are obtained using factory methods like **getByName()**, **getByAddress()**, and **getAllByName()**.

Getter Methods

The **InetAddress** class provides methods to retrieve information.

- **getByName()**: Returns an **InetAddress** object for the specified hostname or IP address.
- **getHostName()**: Returns the hostname.
- **getHostAddress()**: Returns the IP address as a string.
- **getCanonicalHostName()**: Provides the fully qualified domain name (FQDN)

2.2 Address Types, Testing Reachability, and Object Methods

Address Types

- **IPv4:** Standard 32-bit address (e.g. 192.168.0.1).
- **IPv6:** Newer 128-bit address (e.g. 2001:0db8:85a3::8a2e:0370:7334).

Testing Reachability: The `isReachable()` method tests if the system can reach a specific address.

Example:

```
import java.net.InetAddress;

public class ReachabilityTest {
    public static void main(String[] args) throws Exception {
        InetAddress address = InetAddress.getByName("www.google.com");
        if (address.isReachable(5000)) { // Timeout in milliseconds
            System.out.println("Host is reachable");
        } else {
            System.out.println("Host is not reachable");
        }
    }
}
```

Object Methods

- **Equals(Object obj):** Compares two **InetAddress** objects.
- **hashCode():** Returns a hash code for the IP address.
- **toString():** Provides a string representation.

Example:

```
import java.net.InetAddress;
```

```
public class InetAddressMethods {  
    public static void main(String[] args) throws Exception {  
        // Create an InetAddress object
```

Object Methods

```
InetAddress address = InetAddress.getByName("www.google.com");
```

```
// Using object methods
```

```
System.out.println("Host Name: " + address.getHostName()); // Example: www.google.com
```

```
System.out.println("IP Address: " + address.getHostAddress()); // Example: 142.250.190.78
```

```
System.out.println("String Representation: " + address.toString()); // Example:  
www.google.com/142.250.190.78
```

```
// Check equality with another InetAddress object
```

```
InetAddress anotherAddress = InetAddress.getByName("142.250.190.78");
```

```
System.out.println("Is Equal: " + address.equals(anotherAddress)); // true or false
```

```
}
```

```
}
```

2.3 Inet4Address and Inet6Address

These are subclasses of InetAddress:

- **Inet4Address**: Represent IPv4 address.
- **Inet6Address**: Represents IPv6 address.

Example:

```
import java.net.*;
```

```
public class Inet4and6Example {  
    public static void main(String[] args) throws Exception {
```

2.3 Inet4Address and Inet6Address

```
// InetAddress address = Inet4Address.getByName("www.google.com");  
  
    // System.out.println("IPv4 Address: " + address.getHostAddress());  
    // System.out.println("Is IPv4: " + (address instanceof Inet4Address));  
    InetAddress address = Inet6Address.getByName("ipv6.google.com");  
  
    System.out.println("IPv6 Address: " + address.getHostAddress());  
    System.out.println("Is IPv6: " + (address instanceof Inet6Address));  
  
    }  
}
```

2.4 The `NetworkInterface` Class

- The **`NetworkInterface`** class represents network interface (e.g. Ethernet or Wi-Fi), both software as well as hardware, its name, list of IP addresses assigned to it, and all related information.
- It provides methods to query details about network interface, such as its name, address, and hardware (MAC) address.

Factory Methods

- **`NetworkInterface.getByName(String name)`**: Gets a network interface by name.
- **`NetworkInterface.getByInetAddress(InetAddress addr)`**: Gets the interface for a specific IP address.
- **`NetworkInterface.getNetworkInterfaces()`**: Returns an enumeration of all network interface.

2.4 The NetworkInterface Class

Getter Methods

- **getName()**: Returns the interface name (e.g. eth0 or wlan0).
- **getInetAddresses()**: Returns an enumeration of associated IP addresses.
- **getHardwareAddress()**: Retrieves the MAC address.

Example:

```
import java.net.NetworkInterface;  
import java.net.InetAddress;
```

```
import java.util.Enumeration;

public class NetworkInterfaceExample {
    public static void main(String[] args) throws Exception {
        Enumeration<NetworkInterface> interfaces = NetworkInterface.getNetworkInterfaces();

        while (interfaces.hasMoreElements()) {
            NetworkInterface netInterface = interfaces.nextElement();
            System.out.println("Name: " + netInterface.getName());
            System.out.println("Display Name: " + netInterface.getDisplayName());

            Enumeration<InetAddress> addresses = netInterface.getInetAddresses();
            while (addresses.hasMoreElements()) {
                InetAddress address = addresses.nextElement();
                System.out.println("Address: " + address);
            }
        }
    }
}
```

2.5 Some Useful Programs

- **SpamCheck**: A program to check if a specific IP address belongs to a known spammer or is blacklisted. This can be done by querying a DNS-based blacklist (DNSBL).
- Example:

```
import java.net.InetAddress;
```

```
public class SpamCheck {  
    public static void main(String[] args) {
```

```
String spamServer = "spam.example.com"; // Example spam domain
```

```
    try {  
        InetAddress address = InetAddress.getByName(spamServer);  
        System.out.println("This might be a spam source: " +  
address.getHostAddress());  
    } catch (Exception e) {  
        System.out.println("This is likely safe; no spam detected.");  
    }  
}  
}
```

- **Processing Web Server Logfiles:** Web server logs often contain IP addresses. A program can be created to extract and process these addresses (e.g. counting unique visitors).
- Example:

```
import java.io.*;
```

```
public class LogfileProcessor {  
    public static void main(String[] args) {  
        String logFile = "server.log"; // Replace with your actual log file name  
  
        try (BufferedReader br = new BufferedReader(new FileReader(logFile))) {
```

```
String line;
    while ((line = br.readLine()) != null) {
        if (line.contains("200")) { // Only successful visits
            System.out.println(line);
        }
    }
} catch (IOException e) {
    System.out.println("Error reading log file: " + e.getMessage());
}
}
```